>_ MCMP113 · Week 5

Decision Structures: if-else, else-if and switch-case

Choosing what runs: conditions, ladders, switch, input validation and boundary testing

Asst. Prof. Hasan Oğuz, PhD
Introduction to Basic Algorithms and Programming · Istanbul Okan University
Vocational School · 2026–2027 Fall

How to use these notes

These notes follow the Week 5 slides and explain every idea in more detail. All programs are in the folder `code/` (one console project per example, e.g. `05_01_PassFail`); the listings below are included directly from those files, and every console window shows a real run (typed input is shown in **amber**). For every program, **find the branches first** and ask: *which input takes this path?* Then run the program with exactly those inputs. The boundary-test tables in these notes show how; the exercises at the end (⋆–⋆⋆⋆) ask you to do the same for your own programs.

Contents

1 From conditions to decisions 2
2 The if statement 3
3 else-if ladders and nested decisions 5
4 The switch statement 15
5 Input validation and testing 20
6 Common mistakes 24
7 Summary 24
8 Glossary (English – Turkish) 25
9 Exercises 25
10 References 28

Learning outcomes

After this week you should be able to:

1. write `if` and `if–else` statements with a `bool` condition and draw the matching flowchart;
2. build an `else if` ladder for ranges (grades, BMI, prices) and put its conditions in the correct order;
3. nest decisions, combine conditions with `&&`, `||`, `!`, and avoid the missing-brace, stray-semicolon and dangling-else bugs;
4. use the `switch` statement (`case`, `break`, `default`, stacked labels) on `int`, `char` and `string` values, and read a switch expression;
5. validate keyboard input with `int.TryParse` / `double.TryParse` and range checks instead of letting the program crash;
6. choose boundary test inputs so that every branch of a program runs at least once.

>_ 1 From conditions to decisions

In Week 1 we saw that every algorithm is built from three control structures: **sequence**, **selection** and **repetition**. Weeks 3 and 4 were pure sequence: every statement ran exactly once, from top to bottom. In Week 4 we also learned to compute `bool` values, but we could not yet *use* them. This week the `bool` goes inside an `if`, and the program starts to choose which statements run.

Definition – selection (decision) structure

A **selection structure** evaluates a **condition** (an expression of type `bool`) and, depending on its value, executes one of several alternative **branches**. Afterwards all branches meet again and execution continues with the next statement. In our flowcharts selection is drawn in **blue** diamonds with exactly two labelled exits (yes/no or true/false).

C# offers four tools for selection, all covered this week: the `if` statement (with optional `else` and `else if` parts), the `switch` statement, the `switch` *expression*, and the conditional operator `?:` from Week 4. Figure 1 shows their flowcharts.

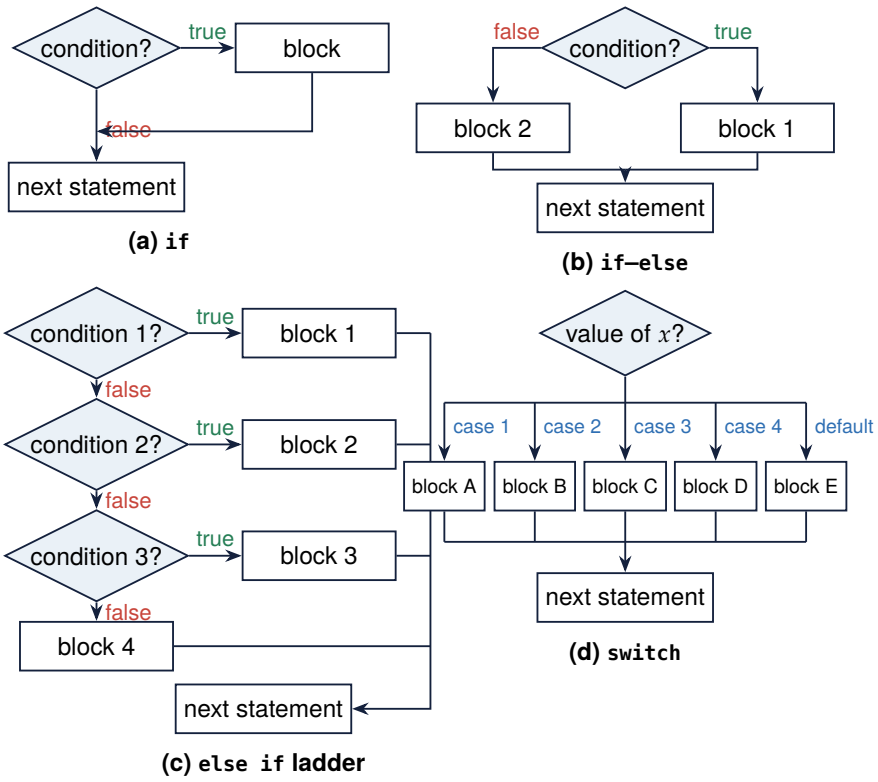


Figure 1: Flowcharts of the four decision patterns. In every case exactly one path is taken, and all paths meet again before the next statement.

>_ 2 The if statement

2.1 One-armed if

Syntax

```
if (condition)    // a bool
{
    // runs only when condition is true
}
```

The keyword `if` is followed by the condition in *parentheses* and a **block** in braces. If the condition is `true`, the block runs; if it is `false`, the block is skipped. In both cases the program continues after the closing brace (Figure 1a).

Example: absolute value (Week 1 algorithm)

```
Console.Write("Enter a number: ");
int x = int.Parse(Console.ReadLine());
if (x < 0)
{
    x = -x;   // runs only when x is negative
}
Console.WriteLine($"Absolute value: {x}");
```

With input -12 the example prints `Absolute value: 12`; with input 5 the condition is false, nothing is changed, and it prints `Absolute value: 5`. (For brevity this snippet uses `int.Parse`; Section 5 shows how to validate the input.)

The condition must be a bool

`if (x = 5)` does not compile: `error CS0029: Cannot implicitly convert type 'int' to 'bool'`. The assignment `x = 5` has the value 5, an `int`, and C# never converts numbers to `bool`. (In C this line compiles and is a classic bug; C# protects you.) Write `x == 5`.

2.2 if-else

With an `else` part, *exactly one* of two blocks runs: the first when the condition is true, the second when it is false (Figure 1b). The pseudocode keywords map one-to-one to C#:

Algorithm 1: pass or fail

```
1 START
2 READ score
3 IF score ≥ 60 THEN
4     PRINT "PASS"
5 ELSE
6     PRINT "FAIL"
7 END IF
8 END
```

```
if (score >= PassMark)
{
    Console.WriteLine($"Score {score}: PASS");
}
else
{
    Console.WriteLine($"Score {score}: FAIL");
}
```

`IF … THEN` becomes `if (…) {`, `ELSE` becomes `} else {`, and `END IF` becomes the closing brace. Remember: in pseudocode ← assigns and = compares; in C# = assigns and == compares. This is the core of Example 1; the complete program (with input validation) follows in Section 5.

2.3 Worked example: even or odd, and the conditional operator

Problem. Read an integer and report whether it is even or odd.

Listing 1: Even or odd (`code/05_02_EvenOdd`)

```
// MCMP113 · Week 5 · Example 2 — Even or odd
// Demonstrates: if-else on a remainder, and the same decision with the ?: operator.
Console.Write("Enter an integer: ");
if (!int.TryParse(Console.ReadLine(), out int n))
{
    Console.WriteLine("Error: that is not a whole number.");
    return;   // ends the program here (a "guard clause")
}

// Version 1: if-else chooses between two ACTIONS.
// Test "== 0", not "== 1": in C#, -7 % 2 is -1.
if (n % 2 == 0)
{
    Console.WriteLine($"{n} is even.");
}
else
{
    Console.WriteLine($"{n} is odd.");
}

// Version 2: the conditional operator ?: chooses between two VALUES.
string kind = (n % 2 == 0) ? "even" : "odd";
Console.WriteLine($"Check with ?: -> {kind}");
```

Run 1

```
Enter an integer: -7
-7 is odd.
Check with ?: -> odd
```

Run 2 (invalid input)

```
Enter an integer: x
Error: that is not a whole number.
```

- **Why `== 0` and not `== 1`?** In C# the remainder has the sign of the dividend (Week 4): `-7 % 2` is -1. The test `n % 2 == 1` would therefore call -7 even. Testing for 0 works for all integers.
- **if-else vs `?:`.** `if–else` chooses between two *actions* (statements); the conditional operator `cond ? a : b` chooses between two *values* and can be used inside an assignment or an interpolated string. Use `?:` only for short two-way values and never nest it deeply.
- **Guard clause.** The first `if` handles bad input, prints a message and executes `return`, which ends a top-level program immediately. The rest of the program can then assume that `n` is valid. We use this pattern in almost every example.

2.4 Blocks and braces

Common mistakes: missing braces (left) and the stray semicolon (right)

```
int balance = 500;
int amount = 150;
if (amount > balance)
    Console.WriteLine("Insufficient funds.");
    Console.WriteLine("Transaction cancelled."
    );
Console.WriteLine("Goodbye.");
```

```
int temperature = 15;
if (temperature > 30);    // empty statement
    !
{
    Console.WriteLine("It is hot!");
}
Console.WriteLine("Done.");
```

Left: without braces, only the *first* statement after `if (…)` belongs to the `if`. The condition is false, yet the program prints `Transaction cancelled.` and `Goodbye.`; the compiler ignores indentation and gives no warning. **Right:** the semicolon right after the condition is an *empty statement*, the whole body of the `if`. The block that follows is an ordinary block that always runs, so the program prints `It is hot!` and `Done.` at 15 degrees. The compiler reports `warning CS0642: Possible mistaken empty statement`. Read warnings, not only errors!

Course rule

Always write braces `{ }`, even when the block has a single statement. Then what you see (indentation) and what runs (blocks) are always the same, and the bugs above and the dangling `else` (Section 3.7) cannot happen.

>_ 3 else-if ladders and nested decisions

3.1 The else-if ladder

```
if (condition1)
{
    // block 1
}
else if (condition2)
{
    // block 2
}
else
{
    // block 3: none was true
}
```

Many problems have more than two cases. Writing `else if` chains several conditions. They are tested **from top to bottom**. As soon as one is true, its block runs and the *rest of the ladder is skipped*; the conditions below it are not even evaluated. The final `else` (optional) catches "none of the above". So in a ladder **exactly one** block runs (or none, if there is no final `else`). Formally, `else if` is not a new keyword: it is an `else` whose single statement is another `if`. In pseudocode we write **ELSE IF** as a shorthand with one **END IF** for the whole ladder.

3.2 Order matters

Because the ladder stops at the *first* true condition, the order of the conditions is part of the logic. Consider a shortened grade scale (AA $\geq$ 90, BB $\geq$ 80, DD $\geq$ 60, otherwise FF) written in the wrong order:

```
if (score >= 60)
{
    letter = "DD";
}
else if (score >= 80)
{
    letter = "BB";   // never reached!
}
else if (score >= 90)
{
    letter = "AA";   // never reached!
}
else
{
    letter = "FF";
}
```

score	wrong order	correct order
95	DD	AA
85	DD	BB
72	DD	DD
60	DD	DD
59	FF	FF

A score of 95 satisfies `score >= 60`, so it gets DD and the ladder stops. The program compiles and runs without any warning; only the test with 95 or 85 reveals the bug. **Rule:** with >= tests start with the highest limit and go down; with < tests start with the lowest limit and go up (BMI example below).

3.3 Worked example: letter grade

Problem. Convert a score 0–100 to a letter grade and grade point using the example scale in the table. (*This is an example scale for our exercises; check the university regulation for the official cut-offs.*)

Score	90–100	85–89	80–84	75–79	70–74	65–69	60–64	0–59
Letter	AA	BA	BB	CB	CC	DC	DD	FF
Point	4.0	3.5	3.0	2.5	2.0	1.5	1.0	0.0

Listing 2: Letter grade (`code/05_05_LetterGrade`)

```
// MCMP113 · Week 5 · Example 5 — Letter grade
// Demonstrates: an else-if ladder, the same logic as a switch expression, and a switch on a string.
// Example scale only; the official cut-offs are in the university regulation.
Console.Write("Enter the score (0-100): ");
if (!int.TryParse(Console.ReadLine(), out int score) || score < 0 || score > 100)
{
    Console.WriteLine("Error: the score must be a whole number from 0 to 100.");
    return;
}

// Version 1: else-if ladder. Order matters: test the highest band first.
string letter;
if (score >= 90)
{
    letter = "AA";
}
else if (score >= 85)
{
    letter = "BA";
}
else if (score >= 80)
{
    letter = "BB";
}
else if (score >= 75)
{
    letter = "CB";
}
else if (score >= 70)
{
    letter = "CC";
}
else if (score >= 65)
{
    letter = "DC";
}
else if (score >= 60)
{
    letter = "DD";
}
else
{
    letter = "FF";
}

// Version 2: switch expression with relational patterns (C# 9). First matching arm wins.
string letter2 = score switch
{
    >= 90 => "AA",
    >= 85 => "BA",
    >= 80 => "BB",
    >= 75 => "CB",
    >= 70 => "CC",
    >= 65 => "DC",
    >= 60 => "DD",
    _ => "FF"
};

// Grade point on the 4-point scale: switch expression on a string.
double point = letter switch
{
    "AA" => 4.0,
    "BA" => 3.5,
    "BB" => 3.0,
```

```
    "CB" => 2.5,
    "CC" => 2.0,
    "DC" => 1.5,
    "DD" => 1.0,
    _ => 0.0
};

Console.WriteLine($"Score {score} -> {letter} (grade point {point:F1})");
Console.WriteLine($"Ladder: {letter}   Switch expression: {letter2}");
```

Output

```
Enter the score (0-100): 85
Score 85 -> BA (grade point 3.5)
Ladder: BA   Switch expression: BA
```

Trace for score = 85. The ladder evaluates `85 >= 90` (false), then `85 >= 85` (true): `letter` becomes `"BA"` and the remaining six conditions are never evaluated. Each `else if` needs only a lower limit, because the upper limit is guaranteed by the earlier false tests: when `score >= 85` is tested, we already know that `score < 90`.

Boundary tests (every row is a real run). A ladder with eight bands has seven internal borders; we test both sides of each, plus the ends of the valid range and one value outside it:

score	letter	score	letter	score	letter	score	letter
100	AA	84	BB	74	CC	64	DD
90	AA	80	BB	70	CC	60	DD
89	BA	79	CB	69	DC	59	FF
85	BA	75	CB	65	DC	0	FF

Inputs `-1`, `101` and `x` all print `Error: the score must be a whole number from 0 to 100.` In all 16 valid runs the ladder and the switch expression (Section 4.5) gave the same letter.

3.4 Worked example: BMI category (ranges on a double)

The body-mass index is BMI $= w/h^2$ (w in kg, h in m). The WHO adult categories are: underweight (BMI < 18.5), normal weight ($18.5 \le$ BMI < 25), overweight ($25 \le$ BMI < 30), obese (BMI ≥ 30). Here the ladder uses < tests, so it starts with the *lowest* limit.

Listing 3: BMI category (`code/05_06_BmiCategory`)

```
// MCMP113 · Week 5 · Example 6 — BMI category
// Demonstrates: range checks with && and an else-if ladder on a double value.
Console.Write("Weight (kg): ");
bool okW = double.TryParse(Console.ReadLine(), out double weight);
Console.Write("Height (m): ");
bool okH = double.TryParse(Console.ReadLine(), out double height);

// Written as "what is VALID": realistic ranges also catch height typed in cm (178).
bool valid = okW && okH && weight > 0 && weight <= 500 && height > 0 && height < 3;
if (!valid)
{
    Console.WriteLine("Error: weight 0-500 kg, height 0-3 m (e.g. 72.5, 1.78).");
    return;
}

double bmi = weight / (height * height);

// WHO adult categories. Each test only needs an upper limit,
// because the lower limit was already excluded by the tests above it.
string category;
if (bmi < 18.5)
```

```
{
    category = "Underweight";
}
else if (bmi < 25)
{
    category = "Normal weight";
}
else if (bmi < 30)
{
    category = "Overweight";
}
else
{
    category = "Obese";
}

Console.WriteLine($"BMI = {bmi:F2} -> {category}");
```

Output

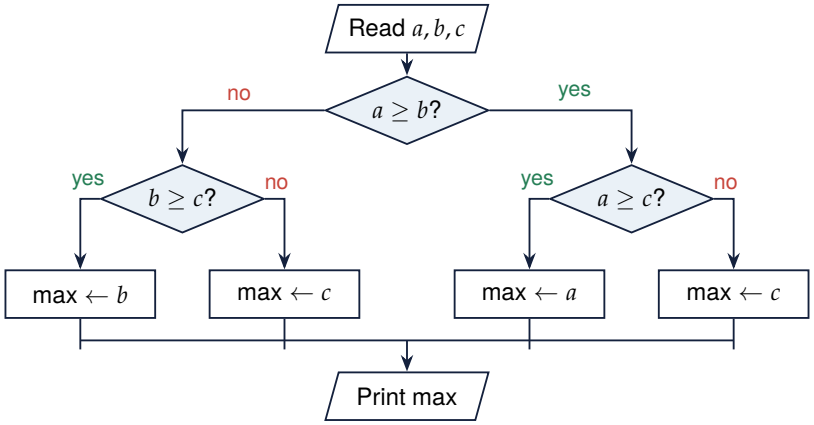
```
Weight (kg): 72.5
Height (m): 1.78
BMI = 22.88 -> Normal weight
```

weight	height	output	what it tests
59.9	1.80	`BMI = 18.49 -> Underweight`	just below 18.5
74	2	`BMI = 18.50 -> Normal weight`	exactly on the border 18.5
100	2	`BMI = 25.00 -> Overweight`	exactly on the border 25
120	2	`BMI = 30.00 -> Obese`	exactly on the border 30
0	1.7	`Error: …`	weight must be positive
72.5	178	`Error: …`	height typed in cm
NaN	1.7	`Error: …`	`double.TryParse` accepts “NaN”

Validation written positively. The condition `valid` describes what a *correct* input looks like. Realistic limits catch the height typed in centimetres ($72.5/178^2 \approx 0.0023$ would otherwise be “underweight”). They also catch the special values `NaN` and `Infinity`, which `double.TryParse` accepts: every comparison with `NaN` is `false`, so `weight > 0` fails. Note also that the category is decided with the unrounded value: a BMI of 18.496 is printed as `18.50` but is still underweight.

3.5 Nested if: the largest of three numbers

A **nested** decision is an `if` inside a branch of another `if`. It is needed when a question depends on the answer to a previous one. To find the largest of a, b, c we first compare a and b, then compare the winner with c:



The flowchart has four paths, so we need at least four tests to run every branch. The program also shows a second, often better method: assume `a` is the largest and let `b` and `c` "challenge" it with two *independent* `if`s (not `else if`: both questions must be asked).

Listing 4: Largest of three (`code/05_03_MaxOfThree`)

```
// MCMP113 · Week 5 · Example 3 — Largest of three numbers
// Demonstrates: nested if-else (method 1) versus a "largest so far" variable (method 2).
Console.Write("a = ");
bool okA = double.TryParse(Console.ReadLine(), out double a);
Console.Write("b = ");
bool okB = double.TryParse(Console.ReadLine(), out double b);
Console.Write("c = ");
bool okC = double.TryParse(Console.ReadLine(), out double c);

if (!okA || !okB || !okC)
{
    Console.WriteLine("Error: please enter three numbers.");
    return;
}

// Method 1: nested if-else. First compare a and b, then the winner with c.
double max1;
if (a >= b)
{
    if (a >= c)
    {
        max1 = a;
    }
    else
    {
        max1 = c;
    }
}
else
{
    if (b >= c)
    {
        max1 = b;
    }
    else
    {
        max1 = c;
    }
}

// Method 2: start with a, then let b and c challenge it.
double max2 = a;
if (b > max2)
{
    max2 = b;
}
if (c > max2)
```

```
{
    max2 = c;
}

Console.WriteLine($"Method 1 (nested if):      {max1}");
Console.WriteLine($"Method 2 (largest so far): {max2}");
```

Output (all numbers negative)

```
a = -2
b = -8
c = -5
Method 1 (nested if):      -2
Method 2 (largest so far): -2
```

a	b	c	a >= b	second test	path	max (both methods)
9	3	5	True	a >= c: True	yes-yes	9
5	3	9	True	a >= c: False	yes-no	9
3	9	5	False	b >= c: True	no-yes	9
3	5	9	False	b >= c: False	no-no	9
7	7	7	True	a >= c: True	yes-yes	7
-2	-8	-5	True	a >= c: True	yes-yes	-2

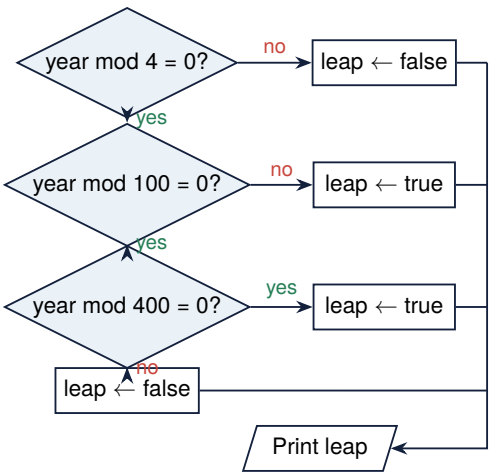
The negative test shows why we never start with `max = 0`: no input is larger than 0, so the answer would be wrong. Method 2 is exactly the idea we will put inside a loop in Week 6 (maximum of N numbers).

Definite assignment

In Method 1 every one of the four paths assigns `max1`. If one branch forgot the assignment, the compiler would refuse to print `max1`: `error CS0165: Use of unassigned local variable 'max1'`. The compiler checks all paths through your decisions.

3.6 Worked example: leap year (nested vs one expression)

Gregorian rule: a year divisible by 4 is a leap year, except years divisible by 100, except years divisible by 400. Asked one question at a time, this is a nested decision:



Algorithm 2: leap year, one expression

1 **START**
2 **READ** *year*
3 $leap \leftarrow (year \bmod 4 = 0$
AND $year \bmod 100 \neq 0)$
OR $year \bmod 400 = 0$
4 **PRINT** *leap*
5 **END**

Listing 5: Leap year (`code/05_04_LeapYear`)

```
// MCMP113 · Week 5 · Example 4 — Leap year
// Demonstrates: the same rule as nested if-else and as one logical expression.
Console.Write("Enter a year: ");
if (!int.TryParse(Console.ReadLine(), out int year) || year < 1)
{
    Console.WriteLine("Error: enter a positive whole number, e.g. 2024.");
    return;
}

// Method 1: nested if-else, one question at a time.
bool leap1;
if (year % 4 == 0)
{
    if (year % 100 == 0)
    {
        if (year % 400 == 0)
        {
            leap1 = true;    // e.g. 2000
        }
        else
        {
            leap1 = false;   // e.g. 1900
        }
    }
    else
    {
        leap1 = true;        // e.g. 2024
    }
}
else
{
    leap1 = false;           // e.g. 2026
}

// Method 2: a single boolean expression (Week 4).
bool leap2 = (year % 4 == 0 && year % 100 != 0)
             || year % 400 == 0;

if (leap1)
{
    Console.WriteLine($"{year} is a leap year (366 days).");
}
else
{
    Console.WriteLine($"{year} is not a leap year (365 days).");
}
Console.WriteLine($"Nested if: {leap1}   Single expression: {leap2}");
```

Output

```
Enter a year: 1900
1900 is not a leap year (365 days).
Nested if: False   Single expression: False
```

year	mod 4 = 0	mod 100 = 0	mod 400 = 0	program output
2026	no	–	–	`2026 is not a leap year (365 days).`
2024	yes	no	–	`2024 is a leap year (366 days).`
1900	yes	yes	no	`1900 is not a leap year (365 days).`
2000	yes	yes	yes	`2000 is a leap year (366 days).`

One test per path: four paths, four years. Inputs `0` and `abc` give the error message of the guard clause. A common error is to join all three tests with &&: `year % 4 == 0 && year % 100 != 0 && year % 400 == 0` is *always* false, because a number divisible by 400 is also divisible by 100.

3.7 The dangling else

What does it print?

```
int x = 5;
int y = -3;
if (x > 0)
    if (y > 0)
        Console.WriteLine("both positive");
else
    Console.WriteLine("x is not positive");
Console.WriteLine("end");
```

Real output

```
x is not positive
end
```

Fixed with braces: prints only "end"

```
if (x > 0)
{
    if (y > 0)
    {
        Console.WriteLine("both positive");
    }
}
else
{
    Console.WriteLine("x is not positive");
}
Console.WriteLine("end");
```

The indentation suggests that the `else` belongs to `if (x > 0)`, but the rule of C# (and C and Java) is: **an `else` is paired with the nearest preceding `if` that does not yet have an `else`**. So it belongs to `if (y > 0)`, and the program claims that $x = 5$ is not positive. With braces the structure is explicit.

3.8 Worked example: triangle type

Problem. Read three whole-number side lengths. Decide whether they form a triangle; if so, classify it as equilateral, isosceles or scalene, and report whether it has a right angle.

Plan. Validate first, then classify: (1) are the inputs whole numbers? (2) is each side in 1..10000? (the upper limit keeps $a \cdot a$ far inside the `int` range); (3) the **triangle inequality**: each side must be shorter than the sum of the other two; (4) the classification ladder; (5) an *independent* right-angle test (Pythagoras: $a^2 + b^2 = c^2$ for some ordering of the sides; with integers the comparison is exact).

Listing 6: Triangle type (`code/05_09_TriangleType`)

```
// MCMP113 · Week 5 · Example 9 — Triangle type
// Demonstrates: validation first, then an else-if ladder, then an independent if.
const int MaxSide = 10000;

Console.Write("Side a: ");
bool okA = int.TryParse(Console.ReadLine(), out int a);
Console.Write("Side b: ");
bool okB = int.TryParse(Console.ReadLine(), out int b);
Console.Write("Side c: ");
bool okC = int.TryParse(Console.ReadLine(), out int c);

if (!okA || !okB || !okC)
{
    Console.WriteLine("Error: the sides must be whole numbers.");
    return;
}
if (a < 1 || b < 1 || c < 1 || a > MaxSide || b > MaxSide || c > MaxSide)
{
    Console.WriteLine($"Error: each side must be between 1 and {MaxSide}.");
    return;
}
// Triangle inequality: each side < sum of the other two.
if (a + b <= c || a + c <= b || b + c <= a)
{
    Console.WriteLine("Not a triangle: one side is too long.");
    return;
}
```

```
28
29  string type;
30  if (a == b && b == c)
31  {
32      type = "equilateral";
33  }
34  else if (a == b || b == c || a == c)
35  {
36      type = "isosceles";
37  }
38  else
39  {
40      type = "scalene";
41  }
42  Console.WriteLine($"Valid triangle: {type}.");
43
44  // Independent question (a separate if, NOT part of the ladder above).
45  if (a * a + b * b == c * c || a * a + c * c == b * b || b * b + c * c == a * a)
46  {
47      Console.WriteLine("It is also a right triangle.");
48  }
```

Run 1

```
Side a: 3
Side b: 4
Side c: 5
Valid triangle: scalene.
It is also a right triangle.
```

a b c	program output (last line)	branch tested
5 5 5	`Valid triangle: equilateral.`	first ladder branch
5 5 8	`Valid triangle: isosceles.`	`a == b`
8 5 5	`Valid triangle: isosceles.`	`b == c`
5 8 5	`Valid triangle: isosceles.`	`a == c`
4 6 7	`Valid triangle: scalene.`	final `else`, no right angle
5 3 4	`It is also a right triangle.`	right angle, hypotenuse first
1 2 3	`Not a triangle: one side is too long.`	$1+2=3$: `<=`, not `<`
0 4 5	`Error: each side must be between 1 and 10000.`	range check
10001 1 1	`Error: each side must be between 1 and 10000.`	upper limit
x 1 1	`Error: the sides must be whole numbers.`	`TryParse` failed

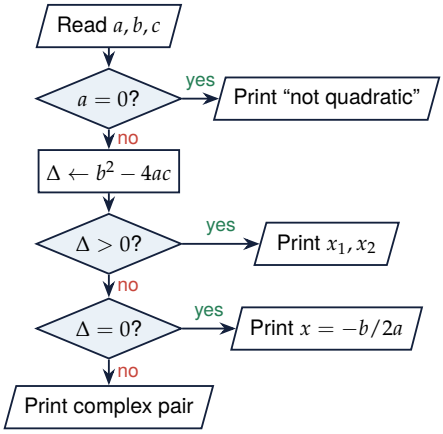
The equilateral test must come *before* the isosceles test: for $5,5,5$ the condition `a == b || b == c || a == c` is also true, so in the opposite order an equilateral triangle would be reported as isosceles.

3.9 Worked example: roots of a quadratic equation

For $ax^2+bx+c=0$ with $a \neq 0$, the discriminant $\Delta = b^2-4ac$ decides the kind of roots:

$$\Delta > 0:\ x_{1,2} = \frac{-b \pm \sqrt{\Delta}}{2a}, \qquad \Delta = 0:\ x = -\frac{b}{2a}, \qquad \Delta < 0:\ x_{1,2} = -\frac{b}{2a} \pm \frac{\sqrt{-\Delta}}{2|a|}\,i.$$

In Week 4 we could only compute Δ and got `NaN` for negative values; now a three-way ladder handles each case, and the case $a=0$ is excluded first (otherwise we would divide by $2a=0$).



a b c	Δ	result printed
1 -3 2	1.00	$x_1 = 2.0000, x_2 = 1.0000$
2 -4 -6	64.00	$x_1 = 3.0000, x_2 = -1.0000$
1 2 1	0.00	$x = -1.0000$
1 2 5	-16.00	$-1.0000 \pm 2.0000\,i$
0 2 1	–	Not a quadratic equation (a = 0).
1 0 0	0.00	$x =$ -0.0000 (see below)

Listing 7: Quadratic roots (code/05_10_QuadraticRoots)

```
// MCMP113 · Week 5 · Example 10 — Roots of a quadratic equation
// Demonstrates: a three-way decision on the discriminant (> 0, == 0, < 0).
Console.WriteLine("Solves a*x^2 + b*x + c = 0");
Console.Write("a = ");
bool okA = double.TryParse(Console.ReadLine(), out double a);
Console.Write("b = ");
bool okB = double.TryParse(Console.ReadLine(), out double b);
Console.Write("c = ");
bool okC = double.TryParse(Console.ReadLine(), out double c);

if (!okA || !okB || !okC)
{
    Console.WriteLine("Error: please enter three numbers.");
    return;
}
if (a == 0)
{
    Console.WriteLine("Not a quadratic equation (a = 0).");
    return;
}

double delta = b * b - 4 * a * c;
Console.WriteLine($"Discriminant = {delta:F2}");

if (delta > 0)
{
    double x1 = (-b + Math.Sqrt(delta)) / (2 * a);
    double x2 = (-b - Math.Sqrt(delta)) / (2 * a);
    Console.WriteLine($"Two real roots: x1 = {x1:F4}, x2 = {x2:F4}");
}
else if (delta == 0)
{
    double x = -b / (2 * a);
    Console.WriteLine($"One real (double) root: x = {x:F4}");
}
else
{
    double re = -b / (2 * a);
    double im = Math.Sqrt(-delta) / (2 * Math.Abs(a));
    Console.WriteLine($"Complex roots: {re:F4} + {im:F4}i and {re:F4} - {im:F4}i");
}
```

$x^2 + 2x + 5 = 0$

```
Solves a*x^2 + b*x + c = 0
a = 1
b = 2
c = 5
Discriminant = -16.00
Complex roots: -1.0000 + 2.0000i and -1.0000 - 2.0000i
```

Checks: $2+1=3=-b/a$ and $2\cdot 1=2=c/a$; for x^2+2x+5: $(-1+2i)^2+2(-1+2i)+5=(1-4i-4)-2+4i+5=0$. `Math.Sqrt` is only ever called with a non-negative argument.

Two subtle points about doubles

(1) `delta == 0` compares doubles exactly. That is fine for whole-number coefficients, but with decimal inputs such as 0.1 rounding errors may make Δ tiny but not zero (Week 4: compare with a tolerance, e.g. `Math.Abs(delta) < 1e-12`). **(2)** For $x^2=0$ ($a=1, b=0, c=0$) the program prints `x = -0.0000`: `-b` is "negative zero", a separate value in IEEE 754 arithmetic that .NET prints with its sign. It compares equal to zero (`-0.0 == 0.0` is `true`), so the result is correct; adding `0.0` (`-b / (2 * a) + 0.0`) would print `0.0000`.

>_ 4 The switch statement

4.1 Syntax and rules

When one value is compared with many *constants*, a `switch` statement is clearer than a ladder of == tests.

```
switch (expression)
{
    case value1:
        // statements
        break;
    case value2:
    case value3:    // stacked labels
        // statements
        break;
    default:        // nothing matched
        // statements
        break;
}
```

- The expression is evaluated **once** and compared with the `case` labels. The labels must be different **constants** of a matching type, e.g. `1`, `'E'`, `"+"` (a variable is not allowed).
- `switch` works with `int`, `char`, `string`, `bool` and other types. String labels are compared exactly: `"x"` and `"X"` are different.
- The matching section runs until `break` (or `return`). `default` runs when no label matches; it is optional but strongly recommended (the "final `else`"); by convention it is last.
- Several labels can be **stacked** on one section, as `value2` and `value3`.

Forgot break: does not compile

```
switch (day)
{
    case 6:
        Console.WriteLine("Saturday");
        // break missing
    case 7:
        Console.WriteLine("Sunday");
        break;
}
```

C# 9 patterns: or, and, relational

```
switch (day)
{
    case 6 or 7:
        Console.WriteLine("weekend");
        break;
    case >= 1 and <= 5:
        Console.WriteLine("weekday");
        break;
    default:
        Console.WriteLine("invalid");
        break;
}
```

No fall-through in C#

In C, C++ and Java a missing `break` silently "falls through" into the next case, a famous source of bugs. C# refuses to compile the left fragment: `error CS0163: Control cannot fall through from one case label ('case 6:') to another.` Every non-empty section must

end with `break`, `return`, `throw` or `goto case`. Empty stacked labels are still allowed.

Patterns in case labels (C# 9). Labels may also be patterns combined with `or`, `and` and relational operators (right fragment). It compiles with the .NET 10 SDK and prints `weekend` for 6, `weekday` for 3 and `invalid` for 9. For the exam, the classic form with stacked labels is enough.

4.2 Worked example: day number to day name

Problem. Read a day number 1..7 (Monday = 1, as in ISO 8601 and in Turkey) and print the name of the day and whether it is a weekday or the weekend.

Listing 8: Day name (`code/05_08_DayName`)

```
// MCMP113 · Week 5 · Example 8 — Day number to day name
// Demonstrates: switch statement on an int, default, return in a case, stacked labels.
Console.Write("Day number (1-7, Monday = 1): ");
if (!int.TryParse(Console.ReadLine(), out int day))
{
    Console.WriteLine("Error: please enter a whole number.");
    return;
}

string name;
switch (day)
{
    case 1:
        name = "Monday";
        break;
    case 2:
        name = "Tuesday";
        break;
    case 3:
        name = "Wednesday";
        break;
    case 4:
        name = "Thursday";
        break;
    case 5:
        name = "Friday";
        break;
    case 6:
        name = "Saturday";
        break;
    case 7:
        name = "Sunday";
        break;
    default:
        Console.WriteLine("Error: the day must be 1-7.");
        return;   // return also ends a case
}

// Stacked labels: 6 and 7 share one block (no fall-through between blocks in C#).
string kind;
switch (day)
{
    case 6:
    case 7:
        kind = "weekend";
        break;
    default:
        kind = "weekday";
        break;
}

Console.WriteLine($"Day {day} is {name} ({kind}).");
```

Run 1

```
Day number (1-7, Monday = 1): 6
Day 6 is Saturday (weekend).
```

Run 2

```
Day number (1-7, Monday = 1): 9
Error: the day must be 1-7.
```

The first `switch` assigns `name` in every case section, and `default` ends the program with `return`; therefore the compiler knows that `name` has a value after the `switch`. The second `switch` uses the stacked labels `case 6: case 7:`. All nine inputs `1`–`7`, `0`, `8` were tested: `1`–`5` print `(weekday)`, `6` and `7 (weekend)`, `0` and `8` the error message; `x` prints `Error: please enter a whole number.`

4.3 Worked example: four-operation calculator (switch on a string)

Listing 9: Calculator (`code/05_07_Calculator`)

```
// MCMP113 · Week 5 · Example 7 — Four-operation calculator
// Demonstrates: switch on a string, stacked case labels, default, and a division-by-zero check.
Console.Write("First number: ");
bool okA = double.TryParse(Console.ReadLine(), out double a);
Console.Write("Operator (+ - * / %): ");
string op = Console.ReadLine();
Console.Write("Second number: ");
bool okB = double.TryParse(Console.ReadLine(), out double b);

if (!okA || !okB)
{
    Console.WriteLine("Error: please enter two numbers.");
    return;
}

switch (op)
{
    case "+":
        Console.WriteLine($"{a} + {b} = {a + b:F2}");
        break;
    case "-":
        Console.WriteLine($"{a} - {b} = {a - b:F2}");
        break;
    case "*":
    case "x":   // stacked labels: *, x and X share one block
    case "X":
        Console.WriteLine($"{a} * {b} = {a * b:F2}");
        break;
    case "/":
        if (b == 0)
        {
            Console.WriteLine("Error: cannot divide by zero.");
        }
        else
        {
            Console.WriteLine($"{a} / {b} = {a / b:F2}");
        }
        break;
    case "%":
        if (b == 0)
        {
            Console.WriteLine("Error: cannot divide by zero.");
        }
        else
        {
            Console.WriteLine($"{a} % {b} = {a % b:F2}");
        }
        break;
    default:
        Console.WriteLine($"Error: unknown operator \"{op}\".");
        break;
}
```

Run 1

```
First number: 7
Operator (+ - * / %): /
Second number: 2
7 / 2 = 3.50
```

Run 2

```
First number: 7
Operator (+ - * / %): /
Second number: 0
Error: cannot divide by zero.
```

Other tested inputs: `7 ^ 2` → `Error: unknown operator "^".`, `7 + 2` → `9.00`, `7 - 2` → `5.00`, `7 * 2`, `7 x 2` and `7 X 2` → `14.00` (stacked labels), `7 % 2` → `1.00`, `7 % 0` → the division error, `-7.5 % 2` → `-1.50` (sign of the dividend). For `double` values C# does *not* throw on division by zero: `7.0 / 0` is `Infinity` and `7.0 % 0` is `NaN` (Week 4). That is why the program checks `b == 0` itself.

4.4 Worked example: shipping cost (ladder + switch on a char)

Problem. An (invented) tariff: base cost 60 TL up to 1 kg, 90 TL up to 5 kg, 150 TL up to 20 kg, and above 20 kg 150 TL plus 8 TL for every *started* kilogram above 20; the express service costs 50 % more. The maximum weight is 100 kg.

$$\text{base}(w) = \begin{cases} 60 & 0 < w \le 1 \\ 90 & 1 < w \le 5 \\ 150 & 5 < w \le 20 \\ 150 + 8\,\lceil w - 20 \rceil & 20 < w \le 100 \end{cases} \qquad \text{total} = \begin{cases} \text{base} & \text{service S} \\ 1.5 \cdot \text{base} & \text{service E} \end{cases}$$

Listing 10: Shipping cost (`code/05_13_ShippingCost`)

```
// MCMP113 · Week 5 · Example 13 — Shipping cost
// Demonstrates: price brackets with an else-if ladder, then a switch on a char.
const decimal ExpressFactor = 1.5m;   // express costs 50 % more

Console.Write("Parcel weight (kg, max 100): ");
if (!decimal.TryParse(Console.ReadLine(), out decimal weight) || weight <= 0 || weight > 100)
{
    Console.WriteLine("Error: the weight must be a number above 0 and at most 100.");
    return;
}
Console.Write("Service (S = standard, E = express): ");
if (!char.TryParse(Console.ReadLine(), out char service))
{
    Console.WriteLine("Error: type a single letter, S or E.");
    return;
}

// Brackets: up to 1 kg, up to 5 kg, up to 20 kg, above 20 kg.
decimal baseCost;
if (weight <= 1)
{
    baseCost = 60m;
}
else if (weight <= 5)
{
    baseCost = 90m;
}
else if (weight <= 20)
{
    baseCost = 150m;
}
else
{
    baseCost = 150m + Math.Ceiling(weight - 20) * 8m;   // 8 TL per started kg above 20
}

decimal total;
string serviceName;
```

```
39 switch (service)
40 {
41     case 'S':
42     case 's':
43         serviceName = "standard";
44         total = baseCost;
45         break;
46     case 'E':
47     case 'e':
48         serviceName = "express";
49         total = baseCost * ExpressFactor;
50         break;
51     default:
52         Console.WriteLine($"Error: unknown service '{service}'.");
53         return;
54 }
55
56 Console.WriteLine($"Base cost: {baseCost:F2} TL");
57 Console.WriteLine($"Total ({serviceName}): {total:F2} TL");
```

Output

```
Parcel weight (kg, max 100): 23.5
Service (S = standard, E = express): e
Base cost: 182.00 TL
Total (express): 273.00 TL
```

Check: $\lceil 23.5 - 20 \rceil = \lceil 3.5 \rceil = 4$, $150 + 4 \cdot 8 = 182$, $1.5 \cdot 182 = 273$. Two decisions run in sequence: the ladder chooses the weight bracket, then the `switch` on a `char` chooses the service. `char.TryParse` succeeds only for exactly one character, so `express` or an empty line is rejected. Money uses `decimal`; `1.5m` is a `decimal` literal. The complete boundary-test plan is homework H4.

4.5 The switch expression (C# 8) and relational patterns (C# 9)

A `switch` *statement* runs statements; a `switch` *expression* produces a *value*, like `?:` but with many arms. Example 5 (Listing of `05_05_LetterGrade`, lines 47–57) computes the letter grade a second time this way: `string letter2 = score switch { >= 90 => "AA", >= 85 => "BA", …, _ => "FF" };`.

- Syntax: `value switch { pattern => result, …};` The arms are separated by commas, and the whole expression ends with a semicolon.
- `>= 90` is a **relational pattern** (C# 9). Patterns can be combined with `and`, `or`, `not`, e.g. `>= 18.5 and < 25`. Constant patterns such as `"AA" => 4.0` work as well (the grade point in Example 5).
- The arms are tried from top to bottom, like a ladder, and the first match wins. `_` (the **discard**) matches everything and plays the role of `default`.
- The compiler checks the arms: an arm that can never be reached is an error (`CS8510: The pattern is unreachable`); this happens with the wrong order `>= 60` before `>= 80`. Without a `_` arm you get `warning CS8509` (the switch expression is not exhaustive), and an unmatched value at run time throws a `SwitchExpressionException`.

The switch expression is presented as a modern, compact alternative. On the exam the `if–else if` ladder is always accepted.

4.6 Which decision structure when?

Structure	Chooses	Best for	Example
if	0 or 1 block	an optional extra step	`if (x < 0) { x = -x; }`
if-else	1 of 2 blocks	two alternatives	`PASS / FAIL`
else-if ladder	1 of many blocks	ranges, mixed conditions	`grades, BMI, prices`
nested if	a path in a tree	a question depends on an answer	`max of three, leap year`
switch statement	1 of many sections	exact values of one variable	`day number, operator`
switch expression	1 of many values	a value from ranges or constants	`score switch { >= 90 => …}`
?:	1 of 2 values	a short two-way value	`n % 2 == 0 ? "even" : "odd"`

There is no "best" structure: choose the one that makes the code easiest to read. Everything can be written with `if`–`else` alone; `switch` and `?:` are conveniences.

>_ 5 Input validation and testing

5.1 TryParse instead of Parse

`int.Parse` crashes on input that is not a number:

```
int.Parse("abc")

Unhandled exception. System.FormatException: The input string 'abc' was not in a correct format.
```

`int.TryParse(text, out int n)` never throws. It *returns* `true` or `false`, and it writes the converted number into the variable `n` given after the keyword `out` (the value is 0 if the conversion failed). The variable is declared right inside the call and can be used afterwards. Lines 3–8 of the even/odd listing (`05_02_EvenOdd`) show the standard pattern: `if (!int.TryParse(Console.ReadLine(), out int n))` prints a message and executes `return` (a guard clause); after the `if`, `n` is a valid `int`.

Call	Returns	Value	Remark
`int.TryParse("42", out n)`	`True`	`42`	
`int.TryParse(" 42 ", out n)`	`True`	`42`	spaces around the number are allowed
`int.TryParse("+7", out n)`	`True`	`7`	leading sign allowed
`int.TryParse("3.5", out n)`	`False`	`0`	not a whole number
`int.TryParse("", out n)`	`False`	`0`	empty line
`int.TryParse("99999999999", out n)`	`False`	`0`	outside the `int` range
`double.TryParse("72,5", out d)`	`True`	`725`	comma = thousands separator!
`double.TryParse("NaN", out d)`	`True`	`NaN`	special value accepted

(All rows were produced by running the calls with the course template.) Our projects use the invariant culture (`InvariantGlobalization`), in which the decimal separator is the point and the comma groups thousands. A Turkish user who types `72,5` therefore gets 725 without any error. This is why **TryParse alone is not enough**: always add a **range check** with realistic limits (Example 6).

5.2 Worked example: pass or fail with validation

In Example 1 the validation itself is part of the ladder: not a number → out of range → pass → fail. The variable `score` declared in the first condition can be used in the following conditions; they are only evaluated when `TryParse` has succeeded.

Listing 11: Pass or fail (`code/05_01_PassFail`)

```
// MCMP113 · Week 5 · Example 1 — Pass or fail
// Demonstrates: if-else, and input validation with int.TryParse + a range check.
const int PassMark = 60;   // example pass mark

Console.Write("Enter the exam score (0-100): ");
string input = Console.ReadLine();

// Each test runs only if all the tests above it were false.
if (!int.TryParse(input, out int score))
{
    Console.WriteLine("Error: please enter a whole number.");
}
else if (score < 0 || score > 100)
{
    Console.WriteLine("Error: the score must be 0-100.");
}
else if (score >= PassMark)
{
    Console.WriteLine($"Score {score}: PASS");
}
else
{
    Console.WriteLine($"Score {score}: FAIL");
}
```

Run 1 (on the pass mark)

```
Enter the exam score (0-100): 60
Score 60: PASS
```

Run 2 (out of range)

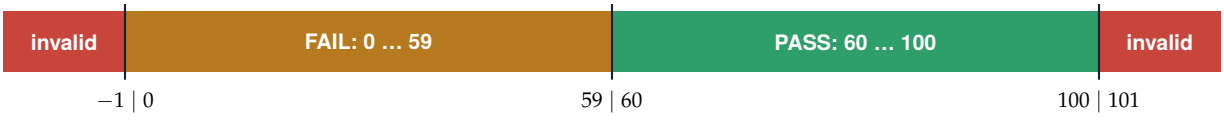
```
Enter the exam score (0-100): 101
Error: the score must be 0-100.
```

Why is the range test before `score >= PassMark`? Otherwise 150 would be reported as PASS. The order of the checks is: (1) is it a whole number? (2) is it in the range? (3) pass or fail?

5.3 Boundary testing

A program with four branches needs at least four tests, but one test per branch is not enough: most bugs sit at the *borders* between branches (> instead of >=, a wrong limit). The method, which extends the Week 1 test-case table:

1. Divide all possible inputs into **classes** that take the same path (invalid text, below range, FAIL, PASS, above range).
2. Choose one typical value inside each class.
3. Add the values on *both sides of every border*: −1/0, 59/60, 100/101.
4. Write down the *expected* output before running; then run and compare.



Test	Input	Expected	Actual output (05_01_PassFail)	OK
not a number	`abc`	error message	`Error: please enter a whole number.`	✓
just below range	`-1`	error message	`Error: the score must be 0-100.`	✓
lower border	`0`	FAIL	`Score 0: FAIL`	✓
just below pass mark	`59`	FAIL	`Score 59: FAIL`	✓
pass mark	`60`	PASS	`Score 60: PASS`	✓
upper border	`100`	PASS	`Score 100: PASS`	✓
just above range	`101`	error message	`Error: the score must be 0-100.`	✓

Ask yourself which bug each test would catch. For example, writing `score > PassMark` instead of `>=` is caught *only* by the test with 60; writing `score > 100` as `score >= 100` is caught only by 100.

5.4 Worked example: ATM withdrawal

This is Week 1 in-class exercise C, now in C#. The order of the ladder *is* the business logic: first reject impossible amounts, then amounts the machine cannot pay out (only 10 TL notes), then amounts the account cannot cover; only then pay.

Listing 12: ATM withdrawal (`code/05_11_AtmWithdrawal`)

```
// MCMP113 · Week 5 · Example 11 — ATM withdrawal (Week 1, exercise C)
// Demonstrates: decimal money values and an else-if ladder whose ORDER encodes the rules.
Console.Write("Account balance (TL): ");
bool okBalance = decimal.TryParse(Console.ReadLine(), out decimal balance);
Console.Write("Amount to withdraw (TL): ");
bool okAmount = decimal.TryParse(Console.ReadLine(), out decimal amount);

if (!okBalance || !okAmount)
{
    Console.WriteLine("Error: please enter numbers only.");
    return;
}

if (amount <= 0)
{
    Console.WriteLine("Refused: amount must be positive.");
}
else if (amount % 10 != 0)
{
    Console.WriteLine("Refused: only multiples of 10 TL.");
}
else if (amount > balance)
{
    Console.WriteLine("Refused: insufficient funds.");
}
else
{
    balance -= amount;
    Console.WriteLine($"Please take your {amount:F2} TL.");
    Console.WriteLine($"New balance: {balance:F2} TL");
}
```

Run (withdraw everything)

```
Account balance (TL): 500
Amount to withdraw (TL): 500
Please take your 500.00 TL.
New balance: 0.00 TL
```

balance	amount	result (real output)
500	200	paid, new balance 300.00
500	500	paid, new balance 0.00
500	510	refused: insufficient funds
500	25	refused: only multiples of 10 TL
500	0	refused: amount must be positive
500	-10	refused: amount must be positive

The border case “withdraw exactly the balance” must be allowed; that is why the test is `amount > balance` and not >=. The remainder operator works for `decimal` too: `25m % 10` is 5.

5.5 Worked example: login check (comparing strings)

Listing 13: Login check (`code/05_12_LoginCheck`)

```
// MCMP113 · Week 5 · Example 12 — Login check (one attempt)
// Demonstrates: comparing strings with ==, combining conditions with && and ||.
const string CorrectUser = "student";
const string CorrectPassword = "Okan2026!";

Console.Write("Username: ");
string user = Console.ReadLine();
Console.Write("Password: ");
string password = Console.ReadLine();
```

```
10
11  // == compares strings exactly: "Student" is not "student".
12  if (user == "" || password == "")
13  {
14      Console.WriteLine("Error: empty username or password.");
15  }
16  else if (user == CorrectUser && password == CorrectPassword)
17  {
18      Console.WriteLine($"Login successful. Welcome, {user}!");
19  }
20  else
21  {
22      // Do not say WHICH part was wrong: it helps attackers.
23      Console.WriteLine("Wrong username or password.");
24  }
```

Run 1

```
Username: student
Password: Okan2026!
Login successful. Welcome, student!
```

Run 2

```
Username: Student
Password: Okan2026!
Wrong username or password.
```

== on strings compares the characters exactly, so it is case-sensitive: `"Student"` is not `"student"`. An empty username or password (just pressing Enter) is caught first. Two security habits: do not tell the user *which* part was wrong (otherwise an attacker learns valid usernames), and never keep real passwords in source code; this is a teaching example. Three attempts need a loop (Week 7).

>_ 6 Common mistakes

Six classic decision bugs

1. **= instead of ==**: `if (x = 5)` → `error CS0029` (`int` is not `bool`). Write `x == 5`.
2. **Chained comparison**: `if (0 <= x <= 100)` → `error CS0019: Operator '<=' cannot be applied to operands of type 'bool' and 'int'`. Write `x >= 0 && x <= 100`.
3. **Stray semicolon**: `if (t > 30);` ends the `if`; the block below always runs (`warning CS0642`).
4. **Missing braces**: only the first statement belongs to the `if`; an `else` pairs with the nearest `if`. Compiles silently.
5. **Missing break**: every case section must end with `break` or `return` (`error CS0163`).
6. **Wrong ladder order**: `>= 60` before `>= 90`, so AA is never reached. Compiles silently; only border tests find it.

Further traps: forgetting to assign a variable in one branch (`error CS0165`); `n % 2 == 1` for negative odd numbers; == on doubles; `double.TryParse("72,5")` giving 725; joining the leap-year tests with && only. Bugs 4 and 6 are the most dangerous ones, because the compiler cannot see them.

>_ 7 Summary

- `if (condition) { …}` runs a block only when the `bool` condition is true; with `else` exactly one of two blocks runs.
- In an `else if` ladder the conditions are tested from top to bottom and the *first* true one wins;

order the conditions so that each one only needs one limit.

- Nested `if`s build a decision tree; every path needs a test. Always use braces: they prevent the missing-brace, stray-semicolon and dangling-`else` bugs.
- `switch` compares one value with constant labels; every section ends with `break` (no fall-through, CS0163); `default` catches the rest; labels can be stacked.
- The switch expression (`value switch { pattern => result, …, _ => …}`) and `?:` choose a *value*; `if`–`else` and the `switch` statement choose *actions*.
- Validate input with `TryParse` (returns `bool`, result via `out`) plus realistic range checks; end the program early with a guard clause.
- Test every branch, and both sides of every border.

>_ 8 Glossary (English – Turkish)

English	Türkçe	English	Türkçe
decision / selection	karar / seçim	condition	koşul
branch	dal	block (of statements)	deyim bloğu
braces { }	süslü parantez	statement	deyim
nested if	iç içe if	else-if ladder	else-if merdiveni
dangling else	sarkan else	empty statement	boş deyim
case label	case etiketi	fall-through	alttaki case'e geçme
default case	varsayılan durum	stacked labels	art arda etiketler
switch expression	switch ifadesi	pattern	desen, örüntü
discard (_)	yok sayma (_)	unreachable	erişilemez
input validation	girdi doğrulama	guard clause	erken çıkış koşulu
parse	ayrıştırmak	out parameter	out parametresi
boundary value	sınır değer	test case	test durumu
equivalence class	denklik sınıfı	range check	aralık kontrolü
equilateral	eşkenar	isosceles	ikizkenar
scalene	çeşitkenar	triangle inequality	üçgen eşitsizliği
discriminant	diskriminant	complex root	karmaşık kök

>_ 9 Exercises

For every programming exercise: write the pseudocode or draw the flowchart first, then the C# program, then a boundary-test table, and run every test. Validate all inputs with `TryParse`. (All solution programs in the instructor edition were compiled with 0 warnings and run with the inputs quoted.)

9.1 In-class exercises

E1 ⋆ Sign of a number

Read an integer and print whether it is positive, negative or zero.

E2 ⋆ Vowel or consonant

Read one letter with `char.TryParse`. Using a `switch` with stacked labels, print whether it is a vowel (a, e, i, o, u, in upper or lower case) or a consonant. Reject anything that is not an English letter.

E3 ⋆⋆ Days in a month

Read a month (1–12) and a year. Print the number of days in that month. Use a `switch` with stacked labels and the leap-year rule for February.

E4 ⋆⋆ Weather advice

Read a temperature in °C (a `double`, valid range -60 to 60) and print advice using a switch expression with relational patterns: below 0 freezing, below 10 cold, below 20 cool, below 30 warm, otherwise hot.

E5 ⋆⋆ Electricity bill

The first 150 kWh cost 2.00 TL each, every further kWh costs 3.00 TL (an invented tariff). Read the consumption (validate ≥ 0) and print the bill.

E6 ⋆⋆⋆ Rock–paper–scissors

Two players type `R`, `P` or `S`. Print `Draw.`, `Player 1 wins.` or `Player 2 wins.` (rock beats scissors, scissors beat paper, paper beats rock). Reject invalid input.

9.2 Homework

H1 ⋆⋆ Parking fee

A car park charges 40 TL for the first hour and 25 TL for every further *started* hour, but at most 250 TL per day. Read the number of minutes parked (1–1440) and print the fee.

H2 ⋆⋆ Minimum and maximum of three

Read three integers. If all three are equal print `All equal: x`; otherwise print the smallest and the largest.

H3 ⋆⋆ Flowchart by hand

Draw the complete flowchart of Example 9 (triangle type) on paper, including all validation steps and the independent right-angle test. Use the symbols of Week 2; every decision has exactly two labelled exits.

H4 ⋆⋆ Test plan

Write a boundary-test table for `05_13_ShippingCost`: one test inside each weight bracket, both sides of every border, both services, and every error message. Write the expected output first, then run every test and record the actual output.

H5 ⋆⋆ Unit converter

Print a menu `1) km -> mile 2) mile -> km 3) kg -> lb 4) lb -> kg`, read the choice and a value, and print the converted value with three decimals. Use a `switch`. Validate both inputs (the value must be 0..1 000 000). Use 1 mile = 1.609344 km and 1 lb = 0.45359237 kg.

9.3 Extra practice

X1 ⋆–⋆⋆ Predict the output

Write down exactly what each snippet prints. Then check by running it.

(a)

```
int a = 7, b = 3;
if (a > 5 && b > 5)
    Console.WriteLine("A");
else if (a > 5 || b > 5)
    Console.WriteLine("B");
else
    Console.WriteLine("C");
```

(b)

```
int n = 15;
if (n % 3 == 0) Console.WriteLine("Fizz");
if (n % 5 == 0) Console.WriteLine("Buzz");
if (n % 3 != 0 && n % 5 != 0)
    Console.WriteLine(n);
```

(c)

```
int k = 2;
switch (k)
{
    case 1:
    case 2:
        Console.WriteLine("small");
        break;
    default:
        Console.WriteLine("big");
        break;
}
```

(d)

```
int t = 25;
string msg = t switch
{
    < 0 => "freezing",
    < 15 => "cold",
    < 25 => "mild",
    _ => "warm"
};
Console.WriteLine(msg);
```

Also: what does (c) print for `k = 1` and `k = 5`?

X2 ⋆⋆ Find the bugs

The following fragment should print “Just adult” for 18, “Adult” for every age ≥ 18, “Baby” for 0 and “Other” otherwise. Find every bug and say whether the compiler reports it.

```
int age = int.Parse(Console.ReadLine());
if (age = 18)
    Console.WriteLine("Just adult");
if (age >= 18);
    Console.WriteLine("Adult");
switch (age)
{
    case 0:
        Console.WriteLine("Baby");
    default:
        Console.WriteLine("Other");
        break;
}
```

X3 ⋆⋆ Ladder to switch expression

Rewrite the BMI ladder of Example 6 as a switch expression with relational patterns. Which inputs must you test to show that both versions agree?

X4 ⋆⋆⋆ Progressive tax

An invented income tax: income up to 10 000 TL is tax-free, the part between 10 000 and 30 000 TL is taxed at 15 %, and the part above 30 000 TL at 25 %. Read the annual income (`decimal`, ≥ 0) and print the tax. Compute the tax for 45 000 TL by hand first.

X5 ⋆⋆ Greeting with patterns

Read an hour 0..23 and print `Good morning!` (5–11), `Good afternoon!` (12–17), `Good evening!` (18–21) or `Good night!` (22–4). Use a `switch` statement with C# 9 patterns such as `case >= 5 and <= 11:`.

>_ 10 References

- Selection statements `if`, `else`, `switch`: https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/statements/selection-statements
- The `switch` expression: https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/operators/switch-expression
- Pattern matching (relational and logical patterns): https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/operators/patterns
- Conditional operator `?:`: https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/operators/conditional-operator
- `Int32.TryParse`: https://learn.microsoft.com/en-us/dotnet/api/system.int32.tryparse
- `Double.TryParse`: https://learn.microsoft.com/en-us/dotnet/api/system.double.tryparse
- `out` parameter modifier: https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/method-parameters
- Tutorial “Branches and loops”: https://learn.microsoft.com/en-us/dotnet/csharp/tour-of-csharp/tutorials/branches-and-loops